Lab2: convolution

September 23, 2026

1 Introduction

In image processing, convolution filtering is a very important operation. A convolution is the operator denoted by $\star$ and defined as follows:

$$f \star g(x) = \int f(x-y)g(y)dy \tag{1}$$

where f and g are two integrable real functions. For functions in $\mathbb{R}^2$, the definition becomes:

$$f \star g(x', y') = \int\int f(x'-x, y'-y)g(x,y)dxdy \tag{2}$$

In the discrete space where images reside, the formula is written:

$$f \star g(k,l) = \sum_i \sum_j f(k-i, l-j)g(i,j) \tag{3}$$

We assume that our functions have a bounded support (i.e., a bounded domain of definition). Thus, the indices i and j vary within a finite interval. More precisely, assuming f is defined on $[1,N]\times[1,P]$ and g is defined on $[-\frac{n}{2},\frac{n}{2}]\times[-\frac{p}{2},\frac{p}{2}]$, the formula (3) becomes:

$$f \star g(k,l) = \sum_{\substack{1 \le k-i \le N \\ -\frac{n}{2} \le i \le \frac{n}{2}}} \sum_{\substack{1 \le l-j \le P \\ -\frac{p}{2} \le j \le \frac{p}{2}}} f(k-i, l-j)g(i,j) \tag{4}$$

This definition is more complex than the previous one. Indeed, convolution involves, among other things, translating one of the two functions. On an infinite support, the translation has no effect on the support. However, with a finite support, the domain is also translated. The consequence is therefore that the support of the convolved function differs from the supports of the convolved functions. In the example of equation 4, the support of $f \star g$ is $[1-\frac{n}{2}, N+\frac{n}{2}] \times [1-\frac{p}{2}, P+\frac{p}{2}]$. Figure 1 illustrates this: the calculation of the pixel in the bottom right corner is possible; its value is a priori non-zero because it depends on the value of a pixel in I (the one in gray), the others being zero.

2 Applications to images

Let I be an image of size $N \times P$, $I(x,y)$ is the luminance value of pixel (x,y). This image can be *convolved* with another image k, which is called the *convolution kernel*. The kernel k is generally small (typically 3×3, 5×5, or 7×7) and often represents a differential operator (but not necessarily); it is represented centered at 0.

For example, if we want to calculate the derivative at x of I ($\frac{\partial I}{\partial x}$), we can approximate by finite difference (first-order Taylor expansions) the operator:

$$\frac{\partial I(x,y)}{\partial x} \sim \frac{1}{2}(I(x+1,y) - I(x-1,y)) \tag{5}$$

which amounts to calculating $I \star k$ with:

$$k = \begin{pmatrix} 0 & 0 & 0 \\ -1/2 & 0 & 1/2 \\ 0 & 0 & 0 \end{pmatrix} \tag{6}$$

With other operators, we can smooth an image, extract contours, etc. Convolution can thus be seen as a weighted averaging operation on a neighborhood fixed by the convolution kernel (see Figure 1).

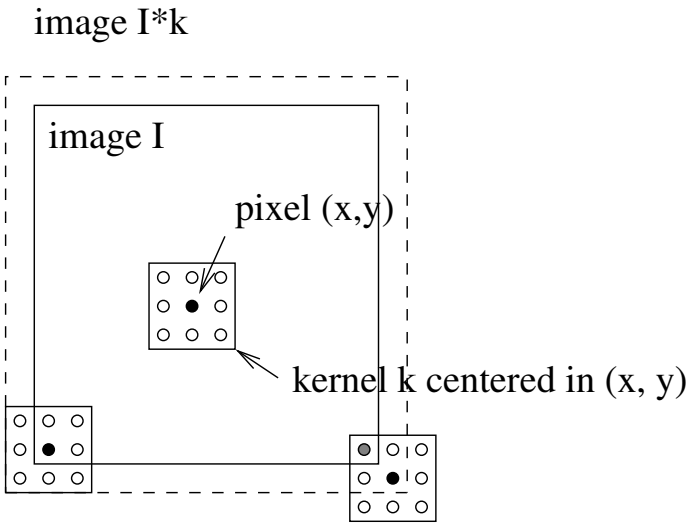


Figure 1: Convolution of I by k: $I \star k(x, y)$ is the weighted sum (by k) of the values of I at (x,y) and their neighbors. This also illustrates that the domain of $I \star k$ (dotted line) is the domain of I augmented by that of k. Beyond this, the values are zero.

3 Sequential algorithm

With the given definition of discrete convolution (equation (4)), we cannot calculate the convolution on the pixels at the edges of the image because it requires access to undefined pixels. However, calculations at the edges are possible (see Figure 1, bottom left and bottom right). To solve this problem, there are two methods:

- either we artificially enlarge the image size, in which case the new pixels have a value of zero;
- or we exclude the pixels at the edges of the image from the convolution calculation.

We choose here the second method because our goal is not to rigorously implement the convolution but to parallelize it.

To achieve long computation times and study the impact of MPI communications, the convolution operation is iterated a large number of times. Therefore, we want to implement the following operation:

$$I \star^n k = I \star \underbrace{k \star k \cdots \star k}_{n \text{ fois}} \tag{7}$$

Since many numerical applications (such as solving partial differential equations) require the repetition of a large number of convolutions, this choice is appropriate.

The final algorithm is rather simple :

1. Reading the command line (via the argv argument);
2. Loading the image (see the read_rasterfile function);
3. Repeat nbiter times the convolution of the image ("pointed to" by the r.data attribute of the variable r of type Raster) using one of the 5 filters:
   (a) memory allocation of an intermediate image buffer,
   (b) for each pixel (i, j) of the image (except the edges), do the following:

- the value of the pixel (i, j) in the buffer image is set to a linear combination (this is the convolution operation) of the pixel (i, j) and its 8 nearest neighbors in the image (r . data);

(c) copying the intermediate buffer into the image (r . data) and freeing the intermediate image buffer from memory;

4. Saving the image (see the save_rasterfile function).

The 5 different linear combinations represent the five possible filters (see the **switch** statement in the code). These operations are, in order: an averaging filter, another averaging filter (with more weight in the center), an edge detector (this is the Laplacian operator discretized by finite differences: such an operator gives a strong response on "peak" type edges, see Figure 2), another edge detector that detects "step" type edges (see Figure 2), and finally a median filter (non-linear: the pixels are sorted according to their brightness and the median value is retained).

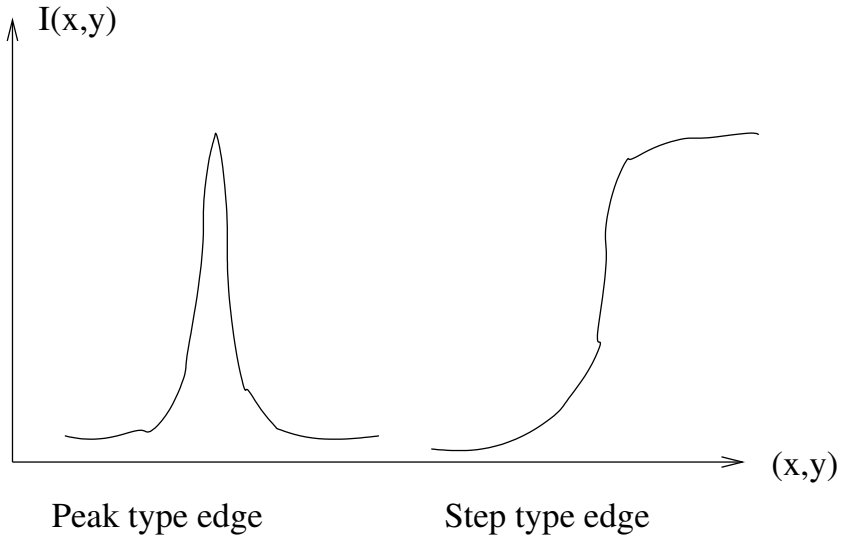


Figure 2: Examples of edge types.

4 MPI work description

1. In the convolution () function, why is an intermediate buffer necessary instead of performing the calculation directly on the image?
2. Which parts of the algorithm can be parallelized?
3. Given that the size of the convolution kernel k is 3×3 pixels, what is the theoretical operation count for processing one pixel of $I \star k$? What type of load balancing is relevant among the MPI processes?
4. What partitioning (data distribution between processes) is natural in this context?
5. What problem (at the edges of the image blocks) arises among the iterations of the convolution operation?
6. Implement a parallel algorithm with blocking MPI communications. Analyze the performance results.
7. Implement a second algorithm using non-blocking MPI communications in order to overlap communications with computations. Analyze the performance results.

5 GPU deployment

5.1 GPU access

We will use the Nvidia P100 GPUs available at Grid'5000. To reserve and access a compute node with such a GPU, you have to run the command:

```
$ oarsub -l /cpu=1/gpu=1/core=2/,walltime=2:0:0 -I -q default -p "gpu_model = 'Tesla P100-PCIE-16
↪ GB'"
```

You should end up on a computer named "chifflot-X".

Run the program:

```
$ /usr/local/cuda/extras/demo_suite/deviceQuery
```

or the command:

```
$ nvidia-smi -a
```

to display the available information on the GPU present in this computer.

We will consider a GPU deployment of a sequential C code (i.e. without MPI). All the code (host code and GPU code) can be written in a single file with the `.cu` suffix and compiled with the CUDA `nvcc` compiler with the following options (for P100 GPUs with *compute capability* 6.0):
`--generate-code arch=compute_60,code=sm_60 -O3.`

To use `nvcc`, you must first load the `cuda-toolkit` module on your current "chifflot" node:

```
$ module load module load cuda-toolkit/12.9.1
```

5.2 GPU work description

We study here the GPU deployment of the same application: the convolution filtering in image processing. We focus on the following averaging filter with a 3×3 convolution kernel:

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 4 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

This corresponds to the second averaging filter (the one with more weight in the center) of the MPI part of the subject.

For pixels at the edges of the image, we will consider that neighboring pixels located outside the image have a value of zero and that the weight for the central pixel corresponds to half the sum of the weights of the neighboring pixels (present in the image). Moreover, the convolution operation will still be iterated multiple times (nbiter times).

Download and read the (new) sequential code in C, the code skeleton in C+CUDA, the images and the Makefiles.

Warning: by default, the image resulting from the last convolution is stored in the `/tmp/your_login` directory of the current node (which usually corresponds to the local hard drive). Please clean this directory at the end of your session, and remind that this directory may be removed at each restart of the node.

---

1. Is such an application well-suited for GPU computing? Justify.
2. Can a single GPU kernel call be used to perform the nbiter iterations on the GPU?
3. Write a CUDA kernel performing this convolution calculation on GPU with 2D thread blocks of size (16, 16), as well as the corresponding host program, in gpu_convol_v0.cu.

4. How to optimize the filling of the computing pipelines and reduce divergence in the instruction flow of threads within the same warp?

5. What are the optimal sizes for the 2D thread blocks?

6. (optional) How to further improve the GPU performance?

7. Although the convolution calculations somewhat differ (at the edges of the image), compare and analyze the performance obtained on multiple CPU cores with MPI (e.g. on `chiclet`) with the one obtained on one P100 GPU on `chifflot` (without taking into account inputs and outputs). Be sure to use the optimization options of the compiler both on CPU (see the lab session on C programming) and on GPU (see the `nvcc` options indicated above).